TaskTracker | SDD As-Built

26/10/2026 | gustavoaraujocd/tasktrackerr | 1.0.0rc1

Identificação e estado

Entrega 3, 26/10/2026. Repositório: https://github.com/gustavoaraujocd/tasktrackerr.

Base: develop, b54839dba345e164e2cfcd8473627a9af96e9733. Candidata 1.0.0rc1 em feature/entrega3-final.

Este documento descreve a implementação candidata; integração em main, homologação humana e release v1.0.0 permanecem pendentes. O manifesto SHA-256 do pacote identifica seus arquivos sem alegar release oficial.

Escopo e arquitetura

Aplicação monousuário em terminal, Python 3.11+, sem servidor HTTP, banco, autenticação ou APIs de rede. A API descrita abaixo é uma interface Python interna.

Fluxo: usuário -> src/main.py -> TaskService -> Task -> dicionário em memória -> terminal.

main.py converte entradas e exibe resultados/erros. models.py valida invariantes. service.py coordena criação, consulta, alteração e busca. O processo mantém um único TaskService; sair descarta todos os dados.

Dados e invariantes

Task é uma dataclass com slots, mutável: title (str obrigatório, trim), description (str, default vazio, trim), priority (Priority, default media), due_date (date ou None), due_time (time ou None), status (Status, default pendente), id (str, UUID4 por padrão), created_at (datetime UTC-aware por padrão).

Priority: baixa, media, alta. Status: pendente, em_andamento, concluida. Apenas instâncias dos enums são aceitas no domínio; a CLI converte strings.

Data rejeita datetime. Horário exige data e não admite tzinfo. created_at não participa da comparação de prazo. is_overdue(reference=None) aceita datetime local sem tzinfo; referências inválidas levantam ValueError.

Prazo sem hora termina em time.max. A comparação é estritamente reference > deadline. Concluídas não atrasam; pendentes e em andamento podem atrasar. Sem data, não há atraso. O relógio padrão é o horário civil local do processo, inclusive no contêiner; ambientes podem ter fusos locais diferentes. A aplicação não converte fusos.

IDs e created_at são preservados pelo serviço ao atualizar. A dataclass permite informar id/created_at diretamente e devolver objetos mutáveis; não há encapsulamento contra mutação por código externo. A CLI usa somente os casos de uso.

Contratos da API Python

TaskService() inicia dict[str, Task] vazio.

create(title, description='', priority=Priority.MEDIUM, due_date=None, due_time=None) -> Task. Valida Task antes de armazenar. Erro de contrato: ValueError.

get(task_id: str) -> Task. ID inexistente: KeyError('task not found'). Retorna a referência armazenada.

list(status: Status | None=None) -> list[Task]. Retorna lista em ordem de inserção, opcionalmente filtrada por igualdade de status. Filtro inválido: ValueError. Lista nova, objetos compartilhados.

overdue(reference: datetime | None=None) -> list[Task]. Aplica is_overdue a cada registro; a validação da referência ocorre em Task.is_overdue. Com cadastro vazio, a referência não é validada.

update(task_id: str, changes) -> Task. Permite title, description, priority, due_date, due_time, status. Primeiro consulta ID; campos não permitidos geram ValueError. dataclasses.replace valida antes de substituir. Em falha, o registro anterior permanece. Remover data exige remover hora no mesmo update.

complete(task_id: str) -> Task equivale a update(status=Status.DONE). Sem restrição adicional de transição: qualquer Status válido pode ser atribuído via update.

Contratos da CLI

1 cadastra; 2 lista; 3 encerra; 4 consulta por ID completo; 5 atualiza; 6 conclui; 7 filtra. Não existe remoção de tarefa.

Cadastro solicita título, descrição, prioridade, data DD/MM/AAAA e, se houver data, hora HH:MM. Título vazio, prioridade e datas/horas inválidas são repetidos. Média ou media são convertidas para o enum.

Atualização: Enter mantém, '-' limpa descrição/data/hora. Título '-' é texto válido. Prioridade/status inválido e data/hora inválidas abortam o formulário, sem salvar campos parciais. ID inexistente informa erro e retorna ao menu.

Filtro vazio informa 'Nenhuma tarefa encontrada para este status.'; listagem vazia informa cadastro vazio (RN17). Alertas aparecem antes da entrada, uma vez por ciclo do menu, sem repetição adicional na listagem (RN18). Ciclos posteriores podem repetir o alerta enquanto houver atraso. EOF/Ctrl+C encerram sem traceback na leitura.

Rastreabilidade de requisitos

RF01 cadastrar, RF02 consultar, RF03 listar, RF04 atualizar, RF05 concluir, RF06 filtrar: TaskService e testes test_service.py/test_cli_workflows.py.

RF07 horário e RF08 atraso: Task, testes de fronteira/validação em test_refinements.py e testes de alertas do menu.

RN01-RN16: docs/especificacao-sdd.md, tests/test_service.py e tests/test_refinements.py.

RN17: test_empty_filter_does_not_claim_tracker_is_empty. RN18: test_overdue_listing_has_one_alert_per_menu_cycle. Origem: apontamentos P2 do PR #17.

RNF01 Python: pyproject.toml; RNF02 harness: pytest/coverage; RNF03 Docker: Dockerfile/Compose; RNF04 modularidade: divisão CLI/domínio; RNF05 governança: PRs com proteção. RNF06-RNF11 de infraestrutura permanecem; Codespaces não foi reconstruído nesta sessão.

Empacotamento e execução

Dockerfile: base python:3.11-slim; WORKDIR /app; COPY src; CMD python src/main.py. Estágio tests instala .[dev] e inclui tests/pyproject. Estágio runtime final usa UID/GID 10001 e não inclui pytest.

Compose: tasktracker-app interativo com stdin/TTY, sem portas ou volume de código; tests separado; coverage grava relatórios por bind mount em reports/. Comandos no README. A imagem usa relógio/fuso do contêiner; adote convenção local consistente para prazos.

Dependências diretas de teste fixadas, mas imagem base, backend e transitivas não estão fixadas por digest/lock completo. Não se promete reprodução binária.

Verificação e limitações

A suíte mede regras, integração e processo real da CLI. Relatório E3 registra ambiente, quantidade e coverage reais. CI Docker é evidência automatizada de execução e não demonstração humana gravada.

Sem armazenamento durável, autorização, concorrência, teste de carga, auditoria ou backup. Não há garantia de uso em produção. Requisitos de aceite humano e vídeo permanecem abertos. Documentos históricos das fases anteriores não são comprovantes desta versão.